UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

PANDUAN PRAKTIKUM PERTEMUAN 7

Polimorfisme: Satu Baris untuk Semua Jenis Alat

Mata Kuliah	Pemrograman Berorientasi Objek · 4 sks · Semester III
Sub-CPMK	Sub-CPMK-7 — Menerapkan polimorfisme melalui duck typing dan pewarisan, menggunakan pernyataan match, serta memahami mengapa Python tidak menyediakan method overloading.
Bobot terhadap nilai akhir	4% — praktikum 1% + Tugas 3 sebesar 3%
Alokasi waktu	Terbimbing 270 menit (kuliah 100 + praktikum 170) · Penugasan terstruktur 205 menit · Belajar mandiri 200 menit
Profil gerbang	p07 — minimal tiga kelas dan ada pewarisan
Bekal dari Pertemuan 6	Repositori bertanda versi p06 dengan hierarki AlatTani dan empat method khusus.
Catatan penting	Ini pertemuan terakhir sebelum Ujian Tengah Semester. Bagian 8 memuat kisi-kisi UTS.
Luaran pertemuan	DaftarHarga polimorfik, penerjemah perintah konsol berbasis match, dan pemformat singledispatch. Bertanda versi p07.

1. PERSOALAN HARI INI

Pada Pertemuan 6 ketiga jenis alat sudah punya cara menghitung sendiri-sendiri. Sekarang UPJA ingin sebuah daftar harga yang memuat seluruh alat. Pertanyaannya sederhana: bagaimana menyusun daftar itu?

Cara yang paling cepat terpikir adalah memeriksa jenis setiap alat lalu menghitung sesuai jenisnya. Cara itu berjalan hari ini. Yang akan Anda buktikan sendiri pada Langkah 2 adalah apa yang terjadi ketika UPJA membeli jenis alat keempat — dan mengapa kerusakannya tidak menimbulkan satu pun galat.

2. TUJUAN DAN INDIKATOR

Setelah menyelesaikan pertemuan ini Anda mampu:

1. Mengenali anti-pola rantai percabangan berdasarkan jenis, dan menjelaskan mengapa ia berbahaya.
2. Menulis kode yang memproses beragam jenis objek tanpa satu pun percabangan berdasarkan jenis.
3. Menjelaskan duck typing beserta kerapuhannya, dan menunjukkan kapan mypy dapat menutupinya.
4. Memakai pernyataan match untuk mencocokkan bentuk data dan perintah, bukan untuk mencocokkan jenis kelas.
5. Memakai functools.singledispatch untuk tipe yang bukan milik sendiri.
6. Menjelaskan mengapa Python tidak menyediakan method overloading, dan apa gantinya.

3. EMPAT CARA, DAN KAPAN MASING-MASING DIPAKAI

Semua yang dibahas hari ini menjawab pertanyaan yang sama: bagaimana memperlakukan hal-hal yang berbeda secara seragam. Yang membedakan adalah keadaannya.

Cara	Dipakai Ketika	Contoh di SIPALA
Polimorfisme lewat pewarisan	Objeknya milik kita dan punya induk yang sama. Ini jawaban baku.	alat.biaya_sewa(3) untuk ketiga jenis alat
Duck typing	Objeknya tidak punya induk yang sama, tetapi menyediakan method yang sama.	Apa pun yang punya biaya_sewa(hari) dapat masuk daftar harga

Cara	Dipakai Ketika	Contoh di SIPALA
Pernyataan match	Yang dibedakan adalah bentuk data atau bentuk perintah, bukan jenis kelas.	Menerjemahkan perintah pengurus: daftar, harga, termurah
functools.singledispatch	Perilaku berbeda menurut tipe argumen, sementara tipenya bukan milik kita.	Memformat int sebagai rupiah dan float sebagai persen

Satu peringatan sejak awal. Pernyataan match sangat menggoda untuk dipakai mencocokkan jenis kelas, karena bentuknya rapi. Jangan. Mencocokkan jenis kelas adalah pekerjaan polimorfisme. Bila Anda menulis match yang cabangnya berupa nama-nama kelas Anda sendiri, Anda sedang menulis ulang rantai if-elif dengan tampilan yang lebih rapi — beserta seluruh kelemahannya.

4. ALUR PERTEMUAN

Waktu	Kegiatan	Bentuk
0–20′	Pembukaan, pembahasan Tugas 2, dan kuis awal	Kuliah interaktif; LKM Bagian A
20–65′	Blok 1: anti-pola percabangan jenis dan polimorfisme	Kuliah dan live coding
65–105′	Blok 2: duck typing, match, dan singledispatch	Kuliah dan demonstrasi
105–115′	Istirahat	—
115–285′	Blok 3: praktikum terbimbing, Langkah 1 sampai 8	Praktikum; LKM Bagian B
285–300′	Blok 4: latihan prediksi keluaran sebagai persiapan UTS	Latihan mandiri; LKM Bagian C
300–320′	Blok 5: kisi-kisi UTS dan penutup	Pengarahan; LKM Bagian D

5. LANGKAH PRAKTIKUM

Berbeda dari biasanya, hari ini kesalahan yang disengaja datang lebih dulu. Anda perlu merasakan persoalannya sebelum melihat jalan keluarnya.

1 10′

Menyiapkan repositori

```
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux atau macOS
git pull
python periksa_pbo.py src --profil p06
python -m pytest
```

2 20′

Kesalahan yang disengaja ①: rantai percabangan jenis

Buat berkas sementara rantai_if.py. Di dalamnya ada satu fungsi yang menghitung biaya dengan memeriksa jenis alat, dan satu kelas alat jenis keempat yang baru dibeli UPJA.

```
def hitung_biaya_cara_lama(alat: AlatTani, jumlah_hari: int) -> int:
    if isinstance(alat, Traktor):
        return alat.tarif_harian * jumlah_hari + 100_000 * jumlah_hari
    elif isinstance(alat, PompaAir):
        solar = (alat.debit * 3) // 100 * 7_000
        return alat.tarif_harian * jumlah_hari + solar * jumlah_hari
    elif isinstance(alat, Perontok):
        return alat.tarif_harian * max(jumlah_hari, 2)
    else:
        return alat.tarif_harian * jumlah_hari      # jenis baru jatuh ke sini


class Penggiling(AlatTani):
    """Jenis alat keempat: penggiling gabah, dibeli UPJA bulan lalu."""

    ONGKOS_ANGKUT: int = 50_000

    def biaya_sewa(self, jumlah_hari: int) -> int:
        return super().biaya_sewa(jumlah_hari) + Penggiling.ONGKOS_ANGKUT
```

Jalankan, lalu bandingkan hasil kedua cara berdampingan:

```
Alat                    cara lama    polimorfik   status
------------------------------------------------------------
Traktor                 Rp750.000     Rp750.000   cocok
Pompa                   Rp603.000     Rp603.000   cocok
Perontok                Rp600.000     Rp600.000   cocok
Penggiling Gabah        Rp360.000     Rp410.000   *** SALAH ***
```

Tidak ada galat. Tidak ada peringatan. Hanya angka yang salah. Cabang else menangkap jenis baru itu diam-diam dan menghitungnya dengan rumus dasar, tanpa ongkos angkut. Petani ditagih lima puluh ribu lebih murah dari seharusnya, dan tidak ada yang tahu sampai pembukuan UPJA tidak cocok di akhir bulan. Perhatikan bahwa cabang else itu justru ditulis dengan niat baik — supaya tidak ada jenis yang tidak tertangani. Niat baik itulah yang menyembunyikan kesalahannya. Catat pada LKM Bagian B.1.

3
25′

Menyusun DaftarHarga tanpa satu pun percabangan jenis

Buat berkas src/model/daftar_harga.py. Perhatikan bahwa tidak ada satu pun nama kelas turunan yang disebut di dalamnya.

```
class DaftarHarga:
    """Menyusun daftar harga tanpa perlu tahu jenis alatnya satu per satu."""

    def __init__(self, alat: list[AlatTani]) -> None:
        if not alat:
            raise ValueError("daftar harga tidak dapat disusun dari daftar kosong")
        self._alat = list(alat)

    def biaya_semua(self, jumlah_hari: int) -> list[tuple[str, int]]:
        return [(alat.nama, alat.biaya_sewa(jumlah_hari)) for alat in self._alat]

    def termurah(self, jumlah_hari: int) -> AlatTani:
        return min(self._alat, key=lambda alat: alat.biaya_sewa(jumlah_hari))

    def total_bila_semua_disewa(self, jumlah_hari: int) -> int:
        return sum(alat.biaya_sewa(jumlah_hari) for alat in self._alat)
```

Perhatikan pula baris self._alat = list(alat). Ia membuat salinan daftar, bukan menyimpan daftar aslinya — penerapan pelajaran Pertemuan 5 tentang komposisi yang tidak bocor. Sekarang tambahkan Penggiling ke dalam daftar dan jalankan lagi. Tidak satu baris pun di dalam DaftarHarga perlu disentuh, dan hasilnya benar.

4
25′

Kesalahan yang disengaja ②: duck typing yang rapuh

Buat berkas sementara bebek_rapuh.py. Di dalamnya ada kelas GerobakSewaan yang bukan turunan AlatTani, tetapi kebetulan punya method bernama sama — dengan tanda tangan berbeda.

```
class GerobakSewaan:
    """Gerobak dorong. Bukan turunan AlatTani, hanya kebetulan mirip."""

    def biaya_sewa(self) -> int:      # TANPA parameter jumlah_hari
        return self._tarif_harian


def cetak_biaya(daftar: list[object], jumlah_hari: int) -> None:
    for barang in daftar:
        biaya = barang.biaya_sewa(jumlah_hari)
```

Keluaran:

```
Traktor                    Rp750.000
Gerobak Dorong                RUNTUH   GerobakSewaan.biaya_sewa() takes
                                       1 positional argument but 2 were given
```

Sekarang lakukan percobaan berikut. Ubah tanda tangan cetak_biaya dari list[object] menjadi list[AlatTani], lalu jalankan mypy tanpa menjalankan programnya.

```
def cetak_biaya(daftar: list[AlatTani], jumlah_hari: int) -> None:

python -m mypy bebek_rapuh.py
→ error: List item 1 has incompatible type "GerobakSewaan";
         expected "AlatTani"  [list-item]
```

Tiga tingkat penjagaan, dan yang ketiga belum kita punya. Duck typing tanpa tipe memberi kelenturan penuh, tetapi kesalahannya baru muncul saat baris itu benar-benar dijalankan — bisa berbulan-bulan kemudian. Menuliskan tipe list[AlatTani] membuat mypy menangkapnya sebelum program dijalankan, tetapi menjadi kaku: gerobak yang tanda tangannya benar pun akan ditolak, semata karena bukan turunan AlatTani. Yang kita butuhkan adalah cara menuntut kontraknya saja tanpa menuntut keturunannya. Itulah typing.Protocol, dan itu materi Pertemuan 9. Catat pada LKM Bagian B.2.

5
30′

Menerjemahkan perintah dengan pernyataan match

Buat folder src/ui beserta berkas src/ui/perintah_konsol.py. Di sinilah pernyataan match dipakai pada tempat yang tepat: mencocokkan bentuk perintah, bukan jenis kelas.

```
    def jalankan(self, perintah: str) -> str:
        match perintah.strip().lower().split():
            case []:
                return "Perintah kosong. Ketik 'bantuan'."

            case ["bantuan"] | ["?"]:
                return PerintahKonsol.BANTUAN

            case ["daftar", hari] if hari.isdigit():
                return self._daftar.tabel(int(hari))

            case ["harga", kode, hari] if hari.isdigit():
                return self._harga_satu_alat(kode.upper(), int(hari))

            case ["termurah", hari] if hari.isdigit():
                alat = self._daftar.termurah(int(hari))
                return f"Termurah untuk {hari} hari: {alat.nama}"

            case [perintah_pertama, *_]:
                return f"Perintah '{perintah_pertama}' tidak dikenal."
```

Empat hal yang perlu Anda perhatikan. Cabang dicocokkan berdasarkan BANYAKNYA bagian perintah, bukan berdasarkan jenis apa pun. Tanda | menyatakan dua bentuk yang diperlakukan sama. Kata if setelah pola disebut penjaga, dan ia menolak perintah yang jumlah harinya bukan angka. Tanda *_ pada cabang terakhir menangkap sisa bagian berapa pun banyaknya, sehingga perintah asing tetap dijawab sopan alih-alih membuat program berhenti.

```
> harga TR-01 3      Traktor Kubota, 3 hari: Rp750.000
> termurah 3         Termurah untuk 3 hari: Perontok Padi
> harga XX-99 1      Alat berkode XX-99 tidak ada dalam daftar.
> daftar 1 2 3       Perintah 'daftar' tidak dikenal. Ketik 'bantuan'.
> ngawur             Perintah 'ngawur' tidak dikenal. Ketik 'bantuan'.
```

6
20′

Perilaku berbeda untuk tipe yang bukan milik kita

Buat berkas src/format_nilai.py. Di sini kita memerlukan perilaku berbeda menurut tipe argumen, sementara tipe-tipe itu — int, float, list — bukan milik kita, sehingga kita tidak bisa menambahkan method ke dalamnya.

```
from functools import singledispatch


@singledispatch
def sebagai_teks(nilai: object) -> str:
    """Bentuk umum, dipakai bila tidak ada bentuk khusus untuk tipe ini."""
    return str(nilai)


@sebagai_teks.register(int)
def _bilangan_bulat(nilai: int) -> str:
    return "Rp" + f"{nilai:,}".replace(",", ".")


@sebagai_teks.register(float)
def _bilangan_pecahan(nilai: float) -> str:
    return f"{nilai * 100:.1f}%"


@sebagai_teks.register(list)
def _daftar(nilai: list[object]) -> str:
    return ", ".join(sebagai_teks(butir) for butir in nilai)
```

Hasilnya: sebagai_teks(150000) menjadi Rp150.000, sebagai_teks(0.825) menjadi 82.5%, dan sebagai_teks([150000, 75000]) menjadi Rp150.000, Rp75.000. Satu nama fungsi, empat perilaku, dipilih Python menurut tipe argumennya.

> **Inilah jawaban Python atas ketiadaan method overloading.** Pada bahasa yang mendukung overloading, Anda menulis beberapa method bernama sama dengan tipe parameter berbeda. Python menolak itu karena namanya cuma satu — yang terakhir menimpa yang sebelumnya. Tetapi perhatikan baik-baik: singledispatch bukan pengganti polimorfisme. Untuk kelas buatan sendiri, jawabannya tetap menambahkan method ke kelasnya. singledispatch hanya dipakai ketika tipe yang dituju bukan milik kita.

7
25′

Menulis pengujian

Minimal tujuh puluh tiga fungsi uji seluruhnya. Yang baru pada pertemuan ini adalah pengujian terhadap perilaku polimorfik dan penerjemah perintah.

```
def test_alat_jenis_baru_langsung_ikut_terhitung() -> None:
    """Menambah jenis alat tidak menuntut perubahan pada DaftarHarga."""

    class Penggiling(AlatTani):
        """Jenis alat keempat yang dibuat khusus untuk pengujian ini."""

        def biaya_sewa(self, jumlah_hari: int) -> int:
            return super().biaya_sewa(jumlah_hari) + 25_000

    daftar = DaftarHarga([*contoh_alat(), Penggiling("PG-01", "Penggiling",
100_000)])
    assert daftar.jumlah_alat == 4


def test_termurah_berubah_untuk_jangka_waktu_berbeda() -> None:
    daftar = DaftarHarga(contoh_alat())
    assert daftar.termurah(1).kode == "PA-01"
    assert daftar.termurah(3).kode == "PR-01"
```

Uji pertama adalah pembuktian polimorfisme yang paling jujur: ia membuat jenis alat baru di dalam badan uji itu sendiri, lalu memastikan DaftarHarga tetap bekerja tanpa diubah. Uji kedua menarik karena hasilnya berbeda menurut jangka waktu — pompa termurah untuk sewa sehari, tetapi perontok yang termurah untuk tiga hari. Itu terjadi karena ketiganya memakai rumus yang berbeda, dan DaftarHarga tidak perlu tahu satu pun di antaranya.

8
15′

Menjalankan ketiga gerbang dan mengumpulkan

```
python periksa_pbo.py src --profil p07     →  LOLOS  (10 kelas)
python -m mypy                              →  Success
python -m pytest                            →  73 passed
python -m src.main

git add .
git commit -m "P07: polimorfisme, match, dan singledispatch"
git tag p07
git push origin main --tags
```

Hapus rantai_if.py dan bebek_rapuh.py sebelum commit.

6. LATIHAN PREDIKSI KELUARAN

Kerjakan pada LKM Bagian C tanpa menjalankan kodenya. Bentuk soal seperti ini akan muncul pada bagian konsep UTS.

1

```
class A:
    def sapa(self) -> str:
        return "A:" + self.nama()
    def nama(self) -> str:
        return "induk"

class B(A):
    def nama(self) -> str:
        return "turunan"

print(B().sapa())
```

Apa keluarannya? Jelaskan method nama() milik siapa yang dipanggil, dan mengapa.

2

```
class Alat:
    def biaya(self, hari: int) -> int:
        return 100 * hari

class Khusus(Alat):
    def biaya(self, hari: int) -> int:
        return super().biaya(hari) + 50

daftar = [Alat(), Khusus(), Alat()]
print(sum(a.biaya(2) for a in daftar))
```

Apa keluarannya? Tunjukkan perhitungannya.

3

```
match "harga TR-01 3".split():
    case ["harga", kode]:
        print("dua bagian")
    case ["harga", kode, hari] if hari.isdigit():
        print("tiga bagian, hari angka")
    case ["harga", *sisa]:
        print("harga dengan sisa", len(sisa))
```

Cabang mana yang dijalankan? Mengapa bukan cabang lainnya?

7. TUGAS 3 (INDIVIDU) — DINILAI TERPISAH

Tugas	Bangun kelas PenyusunLaporan untuk hierarki Pegawai yang Anda buat pada Tugas 2, mengikuti pola DaftarHarga.
Ketentuan utama	Tidak boleh ada satu pun isinstance, type(), maupun match yang mencocokkan nama kelas Anda sendiri di dalam PenyusunLaporan.
Method minimum	Empat method yang seluruhnya memproses list[Pegawai] secara seragam: rekap penghasilan, tertinggi, terendah, dan total.
Pembuktian	Sertakan satu fungsi uji yang membuat jenis pegawai keempat di dalam badan ujinya sendiri, lalu membuktikan PenyusunLaporan tetap bekerja tanpa diubah.
Pernyataan match	Tambahkan satu penerjemah perintah konsol sederhana dengan minimal lima cabang, salah satunya memakai penjaga if dan salah satunya memakai *_.

singledispatch	Tambahkan satu fungsi berdispatch tunggal untuk minimal tiga tipe bawaan Python.
Pengujian	Minimal lima belas fungsi uji, minimal lima membuktikan penolakan atau penanganan perintah asing.
Gerbang	Wajib lolos periksa_pbo.py --profil p07, mypy, dan pytest. Tanda versi p07-tugas3.
Tenggat	Pukul 23.59 pada hari sebelum Ujian Tengah Semester.

8. KISI-KISI UJIAN TENGAH SEMESTER

Bentuk ujian. Ujian berlangsung 120 menit di laboratorium, luring, tanpa akses internet dan tanpa asisten AI. Bobotnya 15% terhadap nilai akhir mata kuliah. Bagian konsep 40%, bagian praktik 60%. Pemeriksa periksa_pbo.py dan mypy tersedia di komputer ujian, dan kode Anda wajib lolos keduanya — kegagalan gerbang mengurangi skor bagian praktik.

8.1 Cakupan dan Bobot

Sub-CPMK	Pokok Bahasan	Bobot	Bentuk Soal
1	Paradigma prosedural dan berorientasi objek; tiga gerbang mutu	5%	Pilihan ganda beralasan
2	Kelas, objek, konstruktor, parameter self, parameter berdefault	15%	Prediksi keluaran; menulis kode
3	Enkapsulasi: property, setter bervalidasi, dataclass beku	20%	Menulis kode; menemukan kesalahan
4	Anggota kelas dan objek; classmethod dan staticmethod; konstanta	15%	Prediksi keluaran; menulis kode
5	Relasi antar kelas; salinan data historis; diagram kelas	15%	Uraian singkat; membaca diagram
6	Pewarisan, super(), penimpaan, empat method khusus	20%	Menulis kode; prediksi keluaran
7	Polimorfisme, duck typing, match, singledispatch	10%	Prediksi keluaran; uraian singkat

8.2 Yang Pasti Ditanyakan

Tujuh butir berikut muncul pada setiap penyelenggaraan UTS mata kuliah ini, dalam bentuk soal yang berbeda-beda.

Butir yang pasti ditanyakan
1. Mengapa konstruktor harus menugaskan lewat setter, dan apa akibatnya bila tidak.
2. Mengapa data historis disalin, bukan dirujuk, beserta akibat lapangannya bila keliru.
3. Perbedaan asosiasi, agregasi, dan komposisi, diterapkan pada kasus yang belum pernah dibahas.
4. Apa yang dikerjakan super().__init__(), dan apa yang runtuh bila ia terlupa.
5. Mengapa __eq__ dan __hash__ tidak boleh dipisah.
6. Membaca kode berisi pewarisan dan penimpaan, lalu memprediksi keluarannya.
7. Mengubah rantai percabangan berdasarkan jenis menjadi kode polimorfik.

8.3 Cara Menyiapkan Diri

1. Baca ulang seluruh LKM Bagian B dari Pertemuan 1 sampai 7. Di sanalah tercatat sepuluh kesalahan yang sengaja ditanam, dan enam di antaranya menjadi dasar soal.
2. Jalankan ulang repositori SIPALA Anda dari awal, lalu jelaskan setiap berkas kepada diri sendiri dengan suara keras.
3. Kerjakan kembali latihan prediksi keluaran pada bagian 6 tanpa melihat jawaban.
4. Pastikan Anda dapat menulis sebuah kelas lengkap dengan konstruktor, property bervalidasi, dan satu method — dari berkas kosong, dalam waktu lima belas menit, tanpa menyalin.

Butir keempat yang paling menentukan. Bagian praktik UTS bernilai 60% dan dikerjakan dari berkas kosong. Mahasiswa yang selama ini menyalin kerangka dari panduan akan kesulitan, bukan karena tidak paham, melainkan karena belum pernah menulisnya sendiri dari nol. Latih itu sebelum hari H.

9. DAFTAR PERIKSA SEBELUM MENINGGALKAN LABORATORIUM

☐	DaftarHarga tidak memuat satu pun isinstance, type(), atau nama kelas turunan.
☐	Pengamatan rantai if-elif yang salah menghitung sudah dicatat pada LKM Bagian B.1.

☐	Percobaan mypy dengan list[AlatTani] sudah dijalankan dan dicatat pada LKM Bagian B.2.
☐	Penerjemah perintah konsol memakai match dengan minimal satu penjaga if dan satu *_.
☐	Berkas format_nilai.py memakai singledispatch untuk minimal tiga tipe bawaan.
☐	Ada uji yang membuat jenis alat baru di dalam badan ujinya sendiri.
☐	Minimal 73 fungsi uji; ketiga gerbang hijau dengan profil p07.
☐	Kedua berkas kesalahan sudah dihapus; tanda versi p07 sudah terdorong.
☐	Latihan prediksi keluaran pada LKM Bagian C sudah dikerjakan tanpa menjalankan kode.
☐	Kisi-kisi UTS sudah dibaca dan dipahami.